

MLE5004: Computer systems architecture

II. Character encoding, integer encoding, signed and unsigned convention.

Contents

2.1	Number Representations	1
2.1.1	Signed numbers	1
2.1.2	Radix complement	2
2.2	2s complement - discussions and examples	2
2.2.1	Bits required to represent a value	6

2.1 Number Representations

A common source for mistakes, when studying for the first time assembly and the architecture of the CPU, is the confusion between numbers and their representation. A beginner may ask “I have a binary number in the **EAX** register; how do I convert that to a hexadecimal number in the **EAX** register?”. The answer is, “You don’t.”.

Although a strong argument could be made that numbers in memory or in registers are represented in binary, it’s best to view values in memory or in a register as abstract numeric quantities. Strings of symbols like 128, 80, or 1000_0000 are not different numbers; they are simply different representations for the same abstract quantity that we refer to as “one hundred twenty-eight”. Inside the computer, **a number is a number regardless of representation**; the only time representation matters is when you input or output the value in a human-readable form.

Human-readable forms of numeric quantities are always strings of characters. To print the value 128 in human-readable form, you must convert the numeric value 128 to the three-character sequence ‘1’ followed by ‘2’ followed by ‘8’. This would provide the decimal representation of the numeric quantity. If you prefer, you could convert the numeric value 128 to the three-character sequence 80h. It’s the same number, but we’ve converted it to a different sequence of characters because (presumably) we wanted to view the number using hexadecimal representation rather than decimal. Likewise, if we want to see the number in binary, then we must convert this numeric value to a string containing a ‘1’ followed by seven ‘0’s.

2.1.1 Signed numbers

Thus far we’ve treated numbers as unsigned values but computers use both positive and negative numbers, and since a computer cannot recognize a plus (+) or a minus (−) sign, an encoding method

must be established to represent the sign of a number in which both positive and negative numbers are distributed as evenly as possible.

There must also be a method to differentiate between positive and negative numbers; that is, there must be an easy way to test the sign of a number. Detection of a number with a zero value must be straightforward. The leftmost (high-order) digit is usually reserved for the sign of the number. Consider the following number A with radix r :

$$A = (a_{n-1}a_{n-2} \dots a_2a_1a_0)_r$$

where digit a_n is the **sign digit** and has the value

$$a_n = \begin{cases} 0 & \text{if } A \geq 0, \\ r - 1 & \text{if } A < 0. \end{cases}$$

The remaining digits of A indicate either the true magnitude or the magnitude in a complemented form. There are three conventional ways to represent positive and negative numbers in a positional number system: sign magnitude, diminished-radix complement, and radix complement. In all three number representations, the high-order digit is the sign of the number, such that, in binary representation: 0 = positive and $r - 1 =$ negative.

We will focus on the *radix complement* representation.

2.1.2 Radix complement

In this representation an integer has the following decimal range:

$$-(r^{n-1}) \text{ to } (r^{n-1} - 1),$$

where the number zero is positive. A positive number A is represented as

$$A = (0a_{n-2}a_{n-3} \dots a_1a_0)_r$$

and a negative number as

$$(A')_{+1} = \{[(r - 1)a'_{n-2}a'_{n-1} \dots a'_1a'_0] + 1\}_r$$

where A' is the diminished-radix complement. Thus, the radix complement is obtained by adding 1 to the diminished-radix complement; that is, $(r - 1) + 1 = r$. Note that all three number representations have the same format for positive numbers and differ only in the way that negative numbers are represented.

In case of the *diminished-radix complement* representation, for the $r - 1$ complement of an individual digit can be determined as $a'_i = (r - 1) - a_i$. For example, if $a_i = 1$ in radix $r = 2$, then the diminished-radix complement of a_i is $a'_i = [(r - 1) - 1] = [(2 - 1) - 1] = 0$. In a similar manner, if $a_i = 0$ in radix $r = 2$, the diminished-radix complement of a_i is $a'_i = [(r - 1) - 0] = [(2 - 1) - 0] = 1$. So, **in binary**, the diminished-radix complement ($r - 1 = 2 - 1 = 1$) is the **1s complement**.

2.2 2s complement - discussions and examples

From now on we will focus on the *2s complement* to represent and interpret integer. Next is an informal discussion over its importance and the difference between representation and interpretation of a numerical value.

Mathematically, the **2s complement representation** of a **negative** number, A , is the value $2^n - |A|$, where $|A|$ represents the absolute value of number A . For the binary number

$$(1101\ 0011)_2 = ((93)_{16} = (147)_{10})$$

in the **UNSIGNED interpretation** its value is $(1101\ 0011)_2 = (147)_{10}$.

Being a binary number with its leftmost digit a 1, in the **SIGNED interpretation**, this number is negative. **What is its value?** Answer: Its value is: $-(2^n - |A|)$.

So, we have to determine the 2s complement of the configuration 10010011. How can we obtain the 2s complement of a number represented in memory (i.e. we are talking about base 2)?

Variants:

1. Subtracting the binary contents of the location from 100...00 (where the number of zero's are exactly the same as the number of digits/bits of the location to be complemented).

$$\begin{array}{r} 1\ 0000\ 0000- \\ 1001\ 0011 \\ \hline 0110\ 1101 \end{array}$$

The result, $(0110\ 1101)_2 = (2^n - A) = (6D)_{16} = 93 + 13 = 109$, the 2s complement on 8 bits of 147 is 109. Thus, the value of $0110\ 1101)_2$ in **SIGNED interpretation** is: $(-109)_{10}$.

2. This variant is derived from the 2s complement definition – faster from a practical point of view: complement the values of all bits of the initial binary number and add 1 to the obtained value.

According to this rule, we start from $(10010011)_2$ and complement the values of all bits, obtaining $(01101100)_2$ after which we add 1 to the obtained value: $(01101100)_2 + (1)_2 = (01101101)_2 = (109)_{10}$.

Thus, the value of $(10010011)_2$ in **SIGNED interpretation** is: $(-109)_{10}$.

3. From a practical point of view this variant is the fastest for obtaining the binary configuration of the 2s complement: keep the low-order bits of 0s and the first bit of 1 unchanged and to complement (invert) the remaining high-order bits.

Applying this rule, we start from $(10010011)_2$ and leave unchanged all low-order bits (the bits starting from the right) of 0 and the first bit 1 (in our example: the first bit of 1 starting from the right – which is the only bit that is left unchanged) and all other bits will be complemented. We obtain $(01101101)_2 = (6D)_{16} = (109)_{10}$.

Thus, the value of $(10010011)_2$ in **SIGNED interpretation** is: $(-109)_{10}$.

4. This variant is derived from the definition of the 2s complement: the sum of the absolute values of the two complementary values is the cardinal of the set of values representable on that size.

- On 8 bits we can represent 2^8 values = 256 values ($[0, 255]$ values in unsigned interpretation or $[-128, +127]$ in signed interpretation).
- On 16 bits we can represent 2^{16} values = 65536 values ($[0, 65535]$ in unsigned interpretation or $[-32768, +32767]$ in signed interpretation).
- On 32 bits we can represent 2^{32} values = 4.294.967.296.

On 8 bits, the 2s complement of $(10010011)_2 (= (147)_{10})$ is $256 - 147 = 109$, thus the corresponding value in **SIGNED interpretation** for $(10010011)_2$ is: $(-109)_{10}$.

One can ask: Why do we need to study the 2s complement? Is it useful for us as programmers? In which way?

$(10010011)_2 (= (93)_{16} = (147)_{10})$, in **UNSIGNED interpretation** $(10010011)_2 = (147)_{16}$.

- What is the signed interpretation of $(10010011)_2$?
a). 01101101 b). -109 (X) c). 6Dh d). 147
- What is the signed interpretation of $(93)_{16}$?
a). 01101101 b). -109 (X) c). 6Dh d). 147
- What is the signed interpretation of 147 in base 10?
a). 01101101 b). -109 (X) c). 6Dh d). 147 (NONE – this does not make sense – we cannot have DIFFERENT interpretations in base 10 of numbers ALREADY expressed in base 10 – the value 147 IS ALREADY AN INTERPRETATION!!)

$(1\ 0000\ 0000)_2 - (1001\ 0011)_2 = (1001\ 1101)_2 = (6D)_{16} = (109)_{10}$, the 2s complement on 8 bits of 147 is -109 , thus the value of $(1001\ 0011)_2$ in **SIGNED interpretation** is: -109 .

147 and -109 are two complementary values, in the sense that $(10010011)_2 =$ either 147, or -109 **depending** on the interpretation. So the complement of 147 is -109 . *Is it also true the other way around? Is -147 the complement of the 109?*

$\rightarrow (109)_{10} = (01101101)_2$, the 2s complement of $(01101101)_2$ is $(10010011)_2 = (147)_{10}$.

Thus, mathematically, the 2s complement representation of a **NEGATIVE** number is the value $2^n - A$, where A is the absolute value of the represented number. The discussion about the **2s complement makes practical sense ONLY WHEN** we refer to the BINARY REPRESENTATION of a NEGATIVE number from base 10! Or to the SIGNED INTERPRETATION of a binary number starting with 1! That is, **ONLY when we discuss the INTERPRETATION of numbers that in base 2 start with 1 (most significant digit of the representation)**! When we have a binary number starting with 0 (its most significant digit), its INTERPRETATION WILL BE THE SAME in both SIGNED and UNSIGNED, i.e. $(01101101)_2 = (109)_{10}$ IN BOTH INTERPRETATIONS. As a result, the fact that 147 is itself "109's complement of 2 as an absolute value" does not help us at all...

The discussion about complementary values makes sense only if we focus on negative numbers! ex. we start from a representation in base 2 that begins with 1 and we ask ourselves what will be the associated negative number in the SIGNED interpretation! Or, we start from an absolute value (ex. $(109)_{10}$ or $(147)_{10}$) and ask ourselves what is the representation in base 2 for $(-109)_{10}$ or for $(-147)_{10}$!

An approach that starts from a representation in base 2 that begins with 0 and asks **ONLY** what will be the "complementary value of that representation" (without this complementary value being then used for something concrete) does not make sense out of the context of interpreting a SIGNED number as negative.

The fact that there is a range of values for which the interpretation of a binary configuration is the same in both interpretations happens (taking the byte size as an example, i.e. $n = 8$ bits used to represent the number) because $[0, 255] \cap [-128, +127] = [0, +127]$ and thus any number in the range $[0, +127]$ will represent both the signed and the unsigned version of the given binary configuration! This representation in base 2 will be a sequence of bits starting with 0 (the most significant digit), for such a sequence in base 10 its value will be the same (a positive one!) in both interpretations.

Let's see which are the types of questions that we can ask starting from a given binary configuration and thus determine what are the situations in which the need to use the 2s complement arises:

- a. *Question:* If we have a representation of type 0... having the value $+abc$ in unsigned interpretation, what will be the value of this representation in signed interpretation? (conversion from base 2 to base 10)

Answer: The same! A number that begins with 0 in base 2 has the same value in base 10 in both interpretations and is a positive number (e.g. 109 is +109 in both interpretations).

- b. *Question:* If we have a representation of type 0... having the value $+abc$, what will be the binary representation of $-abc$? (base 2 $\rightarrow$ base 2)

Answer: If we consider 109, how is -109 represented in base 2? Only in such a question that "the 2s complement starts to play a role", and the answer is: its **REPRESENTATION** will be "the 2s complement of the initial binary configuration". For the value $(109)_{10} = (01101101)_2$, the 2s complement of $(01101101)_2$ is $(10010011)_2$, so $(-109)_{10} = (10010011)_2$.

As a result we can conclude that the complementary value of an integer that begins with 0, will begin with a 1 (the only exception is the value $(0)_{10}$) and will fit as a complementary value in the SIGNED interpretation on the same representation size as the initial value !! $((-109)_{10}$ is also a byte, similar to $(109)_{10}$).

- c. *Question:* If we have (in binary) a representation of type 1... having the value $+abc$ in unsigned interpretation, what will be the value of this representation in signed interpretation? (base 2 $\rightarrow$ base 10)

Answer: The value is: $-(\text{the 2s complement of the initial binary configuration})$. For our example we have: $(10010011)_2 = (147)_{10}$ (unsigned) $= -(\text{the 2s complement of } (10010011)_2) = -(01101101)_2 = (-109)_{10}$.

- d. *Question:* If we consider $(10010011)_2 = (+147)_{10}$, what is the binary representation for $(-147)_{10}$? In other words, if we have (in binary) a representation of type 1... having the value $+abc$, what will be the binary representation for the value $-abc$? (base 2 $\rightarrow$ base 2)

Answer: The answer is similar to b.: its representation will be "the 2s complement of the initial binary configuration". But, if a number begins with the digit 1 in its binary representation and has the value $+abc$ in the unsigned interpretation, then its negative variant $-abc$ will also have to begin with the digit 1 as its associated binary representation. Complementing a binary value of the form 1... will give naturally a binary value starting with a 0 digit on a representation size identical to the initial one!!! (excepting ONLY the values of the -128, +128, -32768, +32768 etc).

As a result, we conclude that if we start from a representation of the form 1... of value $+abc$ **we cannot** obtain the value $-abc$ **on the same representation size!** *Example:* $(147)_{10} = (10010011)_2$ ($147 \in [0, 255]$), but $(-147)_{10} \notin [-128, +127]$, so $(-147)_{10}$ cannot be represented on a byte, even if $(+147)_{10}$ can be represented on a byte!

So, obtaining the representation in binary of $(-147)_{10}$ starting from $(147)_{10} = (10010011)_2$ is done the following way:

- i. The binary representation of $(147)_{10}$ begins with the digit 1, but we notice that $(-147)_{10} \notin [-128, +127]$, but $(-147)_{10} \in [-32768, +32767]$, thus $(-147)_{10}$ is NOT representable as a byte BUT ONLY as a word!
- ii. On a word, $(147)_{10} = (00000000\ 10010011)_2$ (a binary number beginning with the digit 0) and according to b., we have that $(-147)_{10} = \text{"the 2s complement of the initial binary configuration"}$. The 2s complement of the configuration $(00000000\ 10010011)_2$ is $(11111111\ 01101101)_2$, so $(-147)_{10} = (11111111\ 01101101)_2 = (FF6D)_{16}$.

The 2s complement of the configuration $(0000000010010011)_2$ is $(111111101101101)_2$, so $(-147)_{10} = (111111101101101)_2 = (FF6D)_{16}$.

We can check this: $(111111101101101)_2 = (FF6D)_{16} = (65389)_{10}$ (UNSIGNED), the sum of the absolute values of the two complementary values (in base 10) being $65389 + 147 = 65536$ which is the cardinal of the set of values representable on *one word*, so the above two interpretations of the configuration $(111111101101101)_2$ are correct and consistent!

Let us consider the value *VAL*, **we can conclude that:**

1. if the binary configuration for *VAL* starts with **0**, $-VAL$ will be represented on the same representation size (i.e. the same number of *bytes*) as the original representation, e.g. 109 and -109 are represented on the same representation size - 1 *byte*.
2. if the binary configuration for *VAL* starts with **1**, $-VAL$ will be represented on a representation size larger than the original, double the size as an assembly language data type, e.g. 147 is representable on a byte, -147 representable on a word. (In fact, "on paper", we need only 1 more bit, from the point of view of the "theory" of the complementat rules, $(147)_{10} = (10010011)_2 \rightarrow 8$ bits, $(-147)_{10} = (1\ 01101101)_2 \rightarrow 9$ bits required.

2.2.1 Bits required to represent a value

What is the minimum number of bits needed to represent $(-147)_{10}$ in base 2? In binary, on n bits we can represent 2^n values:

- $[0, 2^n - 1]$ **UNSIGNED** values;
- $[-2^{n-1}, 2^{n-1} - 1]$ **SIGNED** values.

As a **data type** in *assembly*, any value can be represented on a *byte*, *word* or *dword*. Thus, $(-147)_{10} \in [-32768, +32767]$ and accordingly to the above discussion we have $(-147)_{10} = (FF6D)_{16} = (111111101101101)_2$ as a value represented on *one word* = 2 bytes.

Example 1. What is the minimum number of bits on which we can represent the value $(3)_{10}$?

Answer: two bits, $(3)_{10} = (11)_2$

On 2 bits we can represent 2^2 values, either the values $[0, 2^2 - 1] = [0, 3]$ in **UNSIGNED interpretation**, or the values $[-2^{2-1}, 2^{2-1} - 1] = [-2, 1]$ in **SIGNED interpretation**.

Example 2. What is the minimum number of bits on which we can represent the value $(-3)_{10}$?

Answer: tree bits, $(-3)_{10} = (101)_2$.

On 3 bits we can represent 2^3 values, either the values $[0, 2^3 - 1] = [0, 7]$ in **UNSIGNED interpretation**, or the values $[-2^{3-1}, 2^{3-1} - 1] = [-4, 3]$ in **SIGNED interpretation**.

References

1. A Vancea. Courses, (tank you :))
2. Cavanagh, J. (2013). X86 Assembly Language and C Fundamentals (1st ed.). CRC Press.
<https://doi.org/10.1201/b14582>
3. Randall Hyde. (2020). The Art of Assembly Language (2nd Edition). No Starch Press.
4. Daniel Kusswurm. (2023). Modern X86 Assembly Language Programming: 32-bit, 64-bit, SSE, and AVX. Apress.